Laporan Tugas Besar

Nama: Zaidan Kamil Munadi (103032430014)

Muhammad Cheng Ho Pulungan (103032400146)

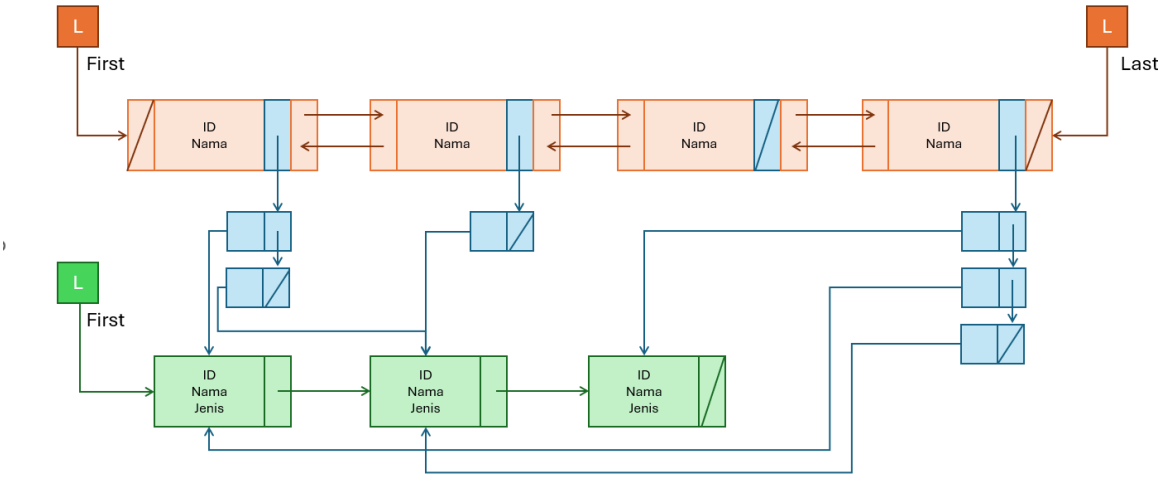
Judul: Manajemen Kepemilikan Data Hewan

Tipe MLL: A

Jenis List Parent: Doubly Linked List

Jenis List Child: Single Linked List

Model MLL:



Data Pemilik Hewan:

- ID (String)
- Nama Pemilik (String)

Data Hewan:

- ID Hewan (String)
- Nama Hewan (String)
- Jenis Hewan (String)

Spesifikasi Program

Program ini merupakan aplikasi berbasis console yang mengelola data Owner (Parent), data Pet (Child), serta Relasi kepemilikan antara Owner dan Pet.

Struktur data yang digunakan:

- **Owner** → Double Linked List (Memiliki pointer prev, next, first, last)
- **Pet** → Single Linked List (Memiliki pointer next, first)
- **Relation** → Single Linked List berisi pointer ke Owner & Pet (Memiliki pointer ownerPtr, petPtr, next, first)

Aplikasi mendukung berbagai operasi CRUD, pencarian relasi, statistik, dan pengurutan data.

Fitur-fitur (berdasarkan menu program)

1. Insert Owner (Parent) Menambahkan data pemilik hewan ke dalam ListOwner. Data yang dicatat: ID Owner, Nama Owner. Program mengecek apakah ID sudah ada, sehingga tidak terjadi duplikasi.

2. Insert Pet (Child) Menambahkan data hewan ke ListPet. Data hewan: ID Pet, Nama Pet, Jenis hewan. Program juga mengecek ID agar tidak terjadi duplikasi.

3. Connect (Hubungkan Owner & Pet) Menghubungkan seorang Owner dengan Pet tertentu. Setiap relasi memungkinkan:

- 1 Owner memiliki banyak hewan
- 1 hewan dapat dimiliki oleh lebih dari satu owner (shared ownership) Program memastikan relasi tidak duplikat sebelum dibuat.

4. Show All Owners Menampilkan seluruh data Owner dalam double linked list.

5. Show All Pets Menampilkan seluruh data hewan yang tersimpan dalam single linked list.

6. Show All Owners beserta Hewannya Menampilkan setiap Owner dan daftar hewan yang ia miliki.

7. Show All Pets beserta Pemiliknya Menampilkan setiap hewan dan siapa saja pemiliknya.

8. Cari Hewan milik Owner tertentu Input: ID Owner. Output: Daftar hewan yang dimiliki owner tersebut.

9. Cari Pemilik dari Hewan tertentu Input: ID Pet. Output: Daftar pemilik dari hewan tersebut.

10. Delete Relation (Putus Hubungan) Menghapus relasi antara Owner dan Pet tertentu tanpa menghapus data masternya.

11. Delete Owner (Cascade Delete) Menghapus Owner sekaligus menghapus semua relasi yang terhubung dengannya secara otomatis. ListOwner diperbarui dengan benar (penanganan hapus awal, akhir, atau tengah).

12. Delete Pet (Cascade Delete) Menghapus seekor hewan dan menghapus semua relasi yang berhubungan dengannya secara otomatis.

13. Statistik (Count) Menampilkan statistik berikut: a. Jumlah Hewan tak bertuan (orphan pets) b. Jumlah Owner tanpa hewan

14. Edit Relation (Pindah Tangan) Mengubah data relasi. Memungkinkan penggantian kepemilikan (ganti Owner) atau penggantian hewan yang dimiliki (ganti Pet) pada sebuah relasi yang sudah ada.

15. Edit Data Owner Mengubah nama Owner berdasarkan ID tertentu.

16. Edit Data Pet Mengubah nama dan jenis Pet berdasarkan ID tertentu.

17. Sort Owner by Name (Ascending) Mengurutkan data Owner berdasarkan nama secara alfabetis (A-Z) menggunakan algoritma Bubble Sort pada Double Linked List.

Persen kontribusi anggota dalam tim (total 100%)

Kontribusi Zaidan Kamil Munadi - 50%

1. **Mendesain struktur data:** a. Struct Owner, Pet, Relation b. Double Linked List Owner c. Single Linked List Pet & Relation
2. **Implementasi fungsi dasar:** a. createListOwner, createListPet, createListRelation b. insertOwner & insertPet c. Fungsi pencarian: findOwner, findPet, findRelation
3. **Menulis fungsi tampil data:** a. printOwners, printPets b. printAllOwnersWithPets
4. **Membuat menu utama & interface awal.**

Kontribusi Muhammad Cheng Ho Pulungan - 50%

1. **Implementasi fitur relasi:** a. connect (menghubungkan Owner-Pet) b. disconnect (hapus relasi)
2. **Mengimplementasikan fitur penghapusan & edit:** a. deleteOwner (cascade delete) b. deletePet (cascade delete) c. editRelation
3. **Menulis fungsi statistik & sorting:** a. countOrphanPets, countChildlessOwners b. sortOwnersByNama
4. **Testing keseluruhan program menggunakan data dummy & debugging.**

BUKTI Repsonsi Bersama Asprak

Lampiran hasil kode 100%

Hewan.h

```
#ifndef HEWAN_H_INCLUDED
#define HEWAN_H_INCLUDED

#include <iostream>
#include <string>
#include <iomanip>

using namespace std;

/* ================================================
   ================ STRUKTUR DATA ==================
   ================================================ */

// -------- DATA --------
struct Owner {
    string id;
    string nama;
};

struct Pet {
    string id;
    string nama;
    string jenis;
};

// -------- POINTER TYPE --------
typedef struct elmOwner* adrOwner;
typedef struct elmPet* adrPet;
typedef struct elmRelation* adrRelation;

// -------- ELEMEN LIST --------

// Parent List (Double Linked List)
struct elmOwner {
    Owner info;
    adrOwner next;
    adrOwner prev;
};

// Child List (Single Linked List)
struct elmPet {
    Pet info;
    adrPet next;
};

// Relation List (Single Linked List)
struct elmRelation {
    adrOwner ownerPtr;
    adrPet petPtr;
    adrRelation next;
};

// -------- LIST --------
struct ListOwner {
    adrOwner first;
    adrOwner last;
};

struct ListPet {
    adrPet first;
};
```

```
struct ListRelation {
    adrRelation first;
};

/* ============================================================
   ================ PRIMITIF DASAR ============================
   ============================================================ */

void createListOwner(ListOwner &L);
void createListPet(ListPet &L);
void createListRelation(ListRelation &L);

adrOwner alokasiOwner(string id, string nama);
adrPet alokasiPet(string id, string nama, string jenis);
adrRelation alokasiRelation(adrOwner O, adrPet P);

/* ============================================================
   ================ INSERT ====================================
   ============================================================ */

void insertOwner(ListOwner &L, adrOwner O);
void insertPet(ListPet &L, adrPet P);
void connect(ListRelation &LR, ListOwner LO, ListPet LP, string idOwner, string idPet);

/* ============================================================
   ================ DELETE ====================================
   ============================================================ */

void deleteOwner(ListOwner &LO, ListRelation &LR, string idOwner);
void deletePet(ListPet &LP, ListRelation &LR, string idPet);
void disconnect(ListRelation &LR, string idOwner, string idPet);

/* ============================================================
   ================ SEARCH ====================================
   ============================================================ */

adrOwner findOwner(ListOwner L, string id);
adrPet findPet(ListPet L, string id);
adrRelation findRelation(ListRelation L, string idOwner, string idPet);

/* ============================================================
   ================ DISPLAY (LOGIC) ===========================
   ============================================================ */

void printOwners(ListOwner L);
void printPets(ListPet L);
void printPetsByOwner(ListRelation LR, string idOwner);
void printOwnersByPet(ListRelation LR, string idPet);
void printAllOwnersWithPets(ListOwner LO, ListRelation LR);
void printAllPetsWithOwners(ListPet LP, ListRelation LR);
```

```
13
14  /* ============================================================
15     ================= DISPLAY (UI TABLE) ======================
16     ============================================================ */
17
18  void printOwnersTable(ListOwner LO);
19  void printPetsTable(ListPet LP);
20  void printOwnersWithPetsTable(ListOwner LO, ListRelation LR);
21  void printPetsWithOwnersTable(ListPet LP, ListRelation LR);
22
23  /* ============================================================
24     ================= COUNT / STAT ============================
25     ============================================================ */
26
27  int countPetsOfOwner(ListRelation LR, string idOwner);
28  int countOwnersOfPet(ListRelation LR, string idPet);
29  int countOrphanPets(ListPet LP, ListRelation LR);
30  int countChildlessOwners(ListOwner LO, ListRelation LR);
31
32  /* ============================================================
33     ================= EDIT / UPDATE ===========================
34     ============================================================ */
35
36  void editRelation(
37      ListRelation &LR,
38      ListOwner LO,
39      ListPet LP,
40      string oldOwnerID,
41      string oldPetID,
42      string newOwnerID,
43      string newPetID
44  );
45
46  void updateOwner(ListOwner &L, string id, string newNama);
47  void updatePet(ListPet &L, string id, string newNama, string newJenis);
48
49  /* ============================================================
50     ================= SORT ====================================
51     ============================================================ */
52
53  void sortOwnersByNama(ListOwner &L);
54
55  #endif // HEWAN_H_INCLUDED
56
57
```

Hewan.cpp

hewan.cpp | hewan.h | main.cpp

```cpp
#include "hewan.h"
void createListOwner(ListOwner &L) {
    L.first = nullptr;
    L.last = nullptr;
}

void createListPet(ListPet &L) {
    L.first = nullptr;
}

void createListRelation(ListRelation &L) {
    L.first = nullptr;
}

adrOwner alokasiOwner(string id, string nama) {
    adrOwner P = new elmOwner;
    P->info.id = id;
    P->info.nama = nama;
    P->next = nullptr;
    P->prev = nullptr;
    return P;
}

adrPet alokasiPet(string id, string nama, string jenis) {
    adrPet P = new elmPet;
    P->info.id = id;
    P->info.nama = nama;
    P->info.jenis = jenis;
    P->next = nullptr;
    return P;
}

adrRelation alokasiRelation(adrOwner P, adrPet C) {
    adrRelation R = new elmRelation;
    R->ownerPtr = P;
    R->petPtr = C;
    R->next = NULL;
    return R;
}
```

```cpp
// ---------------- INSERT (A, B) ----------------
void insertOwner(ListOwner &L, adrOwner P) {
    // Insert Last (Double Linked List)
    if (L.first == nullptr) {
        L.first = P;
        L.last = P;
    } else {
        L.last->next = P;
        P->prev = L.last;
        L.last = P;
    }
}

void insertPet(ListPet &L, adrPet C) {
    // Insert Last (Single Linked List)
    if (L.first == nullptr) {
        L.first = C;
    } else {
        adrPet Q = L.first;
        while (Q->next != nullptr) {
            Q = Q->next;
        }
        Q->next = C;
    }
}

// ---------------- FIND (G, H) ----------------
adrOwner findOwner(ListOwner L, string id) {
    adrOwner P = L.first;
    while (P != nullptr) {
        if (P->info.id == id) return P;
        P = P->next;
    }
    return nullptr;
}

adrPet findPet(ListPet L, string id) {
    adrPet P = L.first;
    while (P != nullptr) {
        if (P->info.id == id) return P;
        P = P->next;
    }
    return nullptr;
}
```

hewan.cpp | hewan.h | main.cpp

```cpp

adrPet findPet(ListPet L, string id) {
    adrPet P = L.first;
    while (P != nullptr) {
        if (P->info.id == id) return P;
        P = P->next;
    }
    return nullptr;
}

adrRelation findRelation(ListRelation L, string idOwner, string idPet) {
    adrRelation R = L.first;
    while (R != nullptr) {
        if (R->ownerPtr->info.id == idOwner && R->petPtr->info.id == idPet) {
            return R;
        }
        R = R->next;
    }
    return nullptr;
}

// ---------------- CONNECT / INSERT RELASI (C) ----------------
void connect(ListRelation &LR, ListOwner LP, ListPet LC, string idOwner, string idPet) {
    adrOwner O = findOwner(LP, idOwner);
    adrPet P = findPet(LC, idPet);

    if (O != NULL && P != nullptr) {
        // Cek apakah sudah ada relasi
        if (findRelation(LR, idOwner, idPet) == nullptr) {
            adrRelation R = alokasiRelation(O, P);
            // Insert First pada Relasi (biar cepat)
            R->next = LR.first;
            LR.first = R;
            cout << "Berhasil menghubungkan " << O->info.nama << " dengan " << P->info.nama << endl;
        } else {
            cout << "Relasi sudah ada." << endl;
        }
    } else {
        cout << "Owner atau Pet tidak ditemukan." << endl;
    }
}

```

```cpp
void disconnect(ListRelation &LR, string idOwner, string idPet) {
    // Menghapus node relasi tertentu
    adrRelation P = LR.first;
    adrRelation Prev = nullptr;
    bool found = false;

    while (P != nullptr && !found) {
        if (P->ownerPtr->info.id == idOwner && P->petPtr->info.id == idPet) {
            found = true;
        } else {
            Prev = P;
            P = P->next;
        }
    }

    if (found) {
        if (Prev == nullptr) { // Hapus elemen pertama
            LR.first = P->next;
        } else {
            Prev->next = P->next;
        }
        delete P;
        cout << "Relasi dihapus." << endl;
    } else {
        cout << "Relasi tidak ditemukan." << endl;
    }
}

void deleteOwner(ListOwner &LP, ListRelation &LR, string idOwner) {
    // 1. Hapus semua relasi yang melibatkan owner ini dulu
    adrRelation R = LR.first;
    while (R != nullptr) {
        adrRelation nextR = R->next;
        if (R->ownerPtr->info.id == idOwner) {
            disconnect(LR, idOwner, R->petPtr->info.id);
        }
        R = R ->next;
    }

    // 2. Hapus Owner dari List Parent
    adrOwner P = findOwner(LP, idOwner);
    if (P != nullptr) {
        if (P == LP.first) { // Hapus awal
            LP.first = P->next;
            if (LP.first != nullptr){
                LP.first->prev = nullptr;
            } else {
                LP.last = nullptr;
            }
        } else if (P == LP.last) { // Hapus akhir
            LP.last = P->prev;
            LP.last->next = nullptr;
        } else { // Hapus tengah
            P->prev->next = P->next;
            P->next->prev = P->prev;
        }
        delete P;
        cout << "Owner berhasil dihapus." << endl;
    } else {
        cout << "Owner tidak ditemukan." << endl;
    }
}
```

```cpp
void deletePet(ListPet &LC, ListRelation &LR, string idPet) {
    // 1. Hapus semua relasi yang melibatkan pet ini
    adrRelation R = LR.first;
    while (R != nullptr) {
        adrRelation nextR = R->next;
        if (R->petPtr->info.id == idPet) {
            disconnect(LR, R->ownerPtr->info.id, idPet);
        }
        R = R ->next;
    }

    // 2. Hapus Pet dari List Child
    adrPet P = findPet(LC, idPet);
    if (P != nullptr) {
        if (P == LC.first) {
            LC.first = P->next;
        } else {
            adrPet Q = LC.first;
            while (Q->next != P) {
                Q = Q->next;
            }
            Q->next = P->next;
        }
        delete P;
        cout << "Pet berhasil dihapus." << endl;
    } else {
        cout << "Pet tidak ditemukan." << endl;
    }
}

// ---------------- SHOW (J, K, L, M, N, O) ----------------
void printOwners(ListOwner L) {
    adrOwner P = L.first;
    cout << "=== List Owner ===" << endl;
    while (P != nullptr) {
        cout << "ID: " << P->info.id << " | Nama: " << P->info.nama << endl;
        P = P->next;
    }
    cout << endl;
}

void printPets(ListPet L) {
    adrPet P = L.first;
    cout << "=== List Hewan ===" << endl;
    while (P != nullptr) {
        cout << "ID: " << P->info.id << " | Nama: " << P->info.nama << " (" << P->info.jenis << ")" << endl;
        P = P->next;
    }
    cout << endl;
}

void printPetsByOwner(ListRelation LR, string idOwner) {
    adrRelation R = LR.first;
    bool found = false;
    cout << "Hewan milik Owner ID " << idOwner << ":" << endl;
    while (R != nullptr) {
        if (R->ownerPtr->info.id == idOwner) {
            cout << "- " << R->petPtr->info.nama << " (" << R->petPtr->info.jenis << ")" << endl;
            found = true;
        }
        R = R->next;
    }
    if (!found) cout << "(Tidak ada hewan)" << endl;
}
```

```cpp

void printAllOwnersWithPets(ListOwner LP, ListRelation LR) {
    adrOwner P = LP.first;
    while (P != nullptr) {
        cout << "Owner: " << P->info.nama << endl;
        printPetsByOwner(LR, P->info.id);
        cout << "-----------------" << endl;
        P = P->next;
    }
}

void printAllPetsWithOwners(ListPet LC, ListRelation LR) {
    adrPet P = LC.first;
    while (P != nullptr) {
        cout << "Hewan: " << P->info.nama << endl;
        printOwnersByPet(LR, P->info.id);
        cout << "-----------------" << endl;
        P = P->next;
    }
}

// ---------------- COUNT (P, Q, R, S) ----------------
int countPetsOfOwner(ListRelation LR, string idOwner) {
    int count = 0;
    adrRelation R = LR.first;
    while (R != nullptr) {
        if (R->ownerPtr->info.id == idOwner) count++;
        R = R->next;
    }
    return count;
}

int countOwnersOfPet(ListRelation LR, string idPet) {
    int count = 0;
    adrRelation R = LR.first;
    while (R != nullptr) {
        if (R->petPtr->info.id == idPet) count++;
        R = R->next;
    }
    return count;
}

int countOrphanPets(ListPet LC, ListRelation LR) {
    int count = 0;
    adrPet P = LC.first;
    while (P != nullptr) {
        if (countOwnersOfPet(LR, P->info.id) == 0) {
            count++;
        }
        P = P->next;
    }
    return count;
}

int countChildlessOwners(ListOwner LP, ListRelation LR) {
    int count = 0;
    adrOwner P = LP.first;
    while (P != nullptr) {
        if (countPetsOfOwner(LR, P->info.id) == 0) {
            count++;
        }
        P = P->next;
    }
    return count;
}
```

```cpp
// ---------------- EDIT (T) ----------------
void editRelation(ListRelation &LR, ListOwner LP, ListPet LC, string oldOwnerID, string oldPetID, string newOwnerID, string newPetID) {
    // Cari relasi lama
    adrRelation R = findRelation(LR, oldOwnerID, oldPetID);
    if (R == nullptr) {
        cout << "Relasi lama tidak ditemukan!" << endl;
        return;
    }

    // Cari Owner dan Pet baru
    adrOwner newOwner = findOwner(LP, newOwnerID);
    adrPet newPet = findPet(LC, newPetID);

    if (newOwner != nullptr && newPet != nullptr) {
        // Cek apakah relasi baru sudah ada
        if (findRelation(LR, newOwnerID, newPetID) == nullptr) {
            R->ownerPtr = newOwner;
            R->petPtr = newPet;
            cout << "Relasi berhasil diubah." << endl;
        } else {
            cout << "Relasi tujuan sudah ada, edit dibatalkan." << endl;
        }
    } else {
        cout << "Data Owner baru atau Pet baru tidak valid." << endl;
    }
}

// ---------------- UPDATE DATA (U, V) ----------------
void updateOwner(ListOwner &L, string id, string newNama) {
    adrOwner P = findOwner(L, id);
    if (P != nullptr) {
        P->info.nama = newNama;
        cout << "Data Owner berhasil diupdate." << endl;
    } else {
        cout << "Owner tidak ditemukan." << endl;
    }
}

void updatePet(ListPet &L, string id, string newNama, string newJenis) {
    adrPet P = findPet(L, id);
    if (P != nullptr) {
        P->info.nama = newNama;
        P->info.jenis = newJenis;
        cout << "Data Pet berhasil diupdate." << endl;
    } else {
        cout << "Pet tidak ditemukan." << endl;
    }
}
```

```cpp
// ---------------- SORTING (W) ----------------
void sortOwnersByNama(ListOwner &L) {
    // Bubble Sort pada Double Linked List (Swap Info)
    if (L.first == nullptr || L.first->next == nullptr) {
            return; // 0 or 1 element
    }

    bool swapped;
    adrOwner P;
    adrOwner LastPtr = nullptr;

    do {
        swapped = false;
        P = L.first;

        while (P->next != LastPtr) {
            if (P->info.nama > P->next->info.nama) {
                // Swap Data
                Owner temp = P->info;
                P->info = P->next->info;
                P->next->info = temp;
                swapped = true;
            }
            P = P->next;
        }
        LastPtr = P;
    } while (swapped);
    cout << "List Owner berhasil diurutkan berdasarkan Nama (A-Z)." << endl;
}
```

Main .cpp

hewan.cpp | hewan.h | main.cpp

```cpp
#include <iostream>
#include <iomanip>
#include <cstdlib>
#include <cctype>
#include "hewan.h"

using namespace std;

/* ================= UTIL UI ================= */

void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

void pauseScreen() {
    cout << "\nTekan ENTER untuk lanjut...";
    cin.ignore();
    cin.get();
}

void printLine(int len = 70) {
    for (int i = 0; i < len; i++) cout << "=";
    cout << endl;
}

/* ================= VALIDATION ================= */

bool isInteger(string s) {
    if (s.empty()) return false;
    for (char c : s) {
        if (!isdigit(c)) return false;
    }
    return true;
}

string getValidOwnerID(ListOwner L) {
    string id;
    while (true) {
        cout << "ID Owner (Angka) : ";
        cin >> id;
        if (!isInteger(id)) {
            cout << "Error: ID harus berupa angka!\n";
        } else if (findOwner(L, id) != NULL) {
            cout << "Error: ID sudah digunakan!\n";
        } else {
            return id;
        }
    }
}
```

hewan.cpp | hewan.h | main.cpp

```cpp
bool isOwnerNameExists(ListOwner L, string nama) {
    adrOwner P = L.first;
    while (P != NULL) {
        if (P->info.nama == nama) {
            return true;
        }
        P = P->next;
    }
    return false;
}

string getValidOwnerName(ListOwner L) {
    string nama;
    while (true) {
        cout << "Nama Owner : ";
        cin >> nama;
        if (isOwnerNameExists(L, nama)) {
            cout << "Error: Nama Owner sudah ada!\n";
        } else {
            return nama;
        }
    }
}

string getValidPetID(ListPet L) {
    string id;
    while (true) {
        cout << "ID Pet (Angka)    : ";
        cin >> id;
        if (!isInteger(id)) {
            cout << "Error: ID harus berupa angka!\n";
        } else if (findPet(L, id) != NULL) {
            cout << "Error: ID sudah digunakan!\n";
        } else {
            return id;
        }
    }
}

```

```
95
96  void showMenu() {
97      printLine();
98      cout << "   APLIKASI MANAJEMEN HEWAN & PEMILIK (MULTI LINKED LIST)\n";
99      printLine();
100     cout << "|  1 | Insert Owner (Parent)                          |\n";
101     cout << "|  2 | Insert Pet (Child)                             |\n";
102     cout << "|  3 | Connect Owner & Pet                            |\n";
103     cout << "|  4 | Show All Owners                                |\n";
104     cout << "|  5 | Show All Pets                                  |\n";
105     cout << "|  6 | Show Owners beserta Hewannya                   |\n";
106     cout << "|  7 | Show Pets beserta Pemiliknya                   |\n";
107     cout << "|  8 | Cari Hewan milik Owner                         |\n";
108     cout << "|  9 | Cari Pemilik dari Hewan                        |\n";
109     cout << "| 10 | Delete Relation                                |\n";
110     cout << "| 11 | Delete Owner (Cascade Delete)                  |\n";
111     cout << "| 12 | Delete Pet (Cascade Delete)                    |\n";
112     cout << "| 13 | Statistik                                      |\n";
113     cout << "| 14 | Edit Relation (Pindah Tangan)                  |\n";
114     cout << "| 15 | Edit Data Owner                                |\n";
115     cout << "| 16 | Edit Data Pet                                  |\n";
116     cout << "| 17 | Sort Owner by Name (Ascending)                 |\n";
117     cout << "|  0 | Exit                                           |\n";
118     printLine();
119     cout << "Pilihan : ";
120 }
121
122 /* ================= MAIN ================= */
123
124 int main() {
125     ListOwner LO;
126     ListPet LP;
127     ListRelation LR;
128
129     createListOwner(LO);
130     createListPet(LP);
131     createListRelation(LR);
132
133     /* ================= DATA DUMMY ================= */
134
135     // OWNER
136     insertOwner(LO, alokasiOwner("1", "Budi"));
137     insertOwner(LO, alokasiOwner("2", "Siti"));
138     insertOwner(LO, alokasiOwner("3", "Andi"));
139
140     // PET
141     insertPet(LP, alokasiPet("1", "Molly", "Kucing"));
142     insertPet(LP, alokasiPet("2", "Bruno", "Anjing"));
143     insertPet(LP, alokasiPet("3", "Tweety", "Burung"));
144     insertPet(LP, alokasiPet("4", "Nemo", "Ikan"));
145
```

```
146     // RELATION
147     connect(LR, LO, LP, "1", "1"); // Budi - Molly
148     connect(LR, LO, LP, "1", "2"); // Budi - Bruno
149     connect(LR, LO, LP, "2", "1"); // Siti - Molly (shared)
150     connect(LR, LO, LP, "3", "4"); // Andi - Nemo
151
152
153     int choice;
154     string choiceStr;
155     string id1, id2, id3, id4, nama, jenis;
156
157     do {
158         clearScreen();
159         showMenu();
160         cin >> choiceStr;
161         cin.ignore();
162
163         if (isInteger(choiceStr)) {
164             choice = atoi(choiceStr.c_str());
165         } else {
166             choice = -1;
167         }
168
169         clearScreen();
170
171         switch (choice) {
172
173         case 1:
174             cout << "--- INSERT OWNER ---\n";
175             // cout << "ID Owner   : "; cin >> id1;
176             id1 = getValidOwnerID(LO);
177             // cout << "Nama Owner : "; cin >> nama;
178             nama = getValidOwnerName(LO);
179             insertOwner(LO, alokasiOwner(id1, nama));
180             pauseScreen();
181             break;
182
183         case 2:
184             cout << "--- INSERT PET ---\n";
185             // cout << "ID Pet : "; cin >> id1;
186             id1 = getValidPetID(LP);
187             cout << "Nama   : "; cin >> nama;
188             cout << "Jenis  : "; cin >> jenis;
189             insertPet(LP, alokasiPet(id1, nama, jenis));
190             pauseScreen();
191             break;
```

```
193         case 3:
194             cout << "--- CONNECT OWNER & PET ---\n";
195             cout << "ID Owner : "; cin >> id1;
196             cout << "ID Pet   : "; cin >> id2;
197             connect(LR, LO, LP, id1, id2);
198             pauseScreen();
199             break;
200 
201         case 4:
202             printOwnersTable(LO);
203             pauseScreen();
204             break;
205 
206         case 5:
207             printPetsTable(LP);
208             pauseScreen();
209             break;
210 
211         case 6:
212             printOwnersWithPetsTable(LO, LR);
213             pauseScreen();
214             break;
215 
216         case 7:
217             printPetsWithOwnersTable(LP, LR);
218             pauseScreen();
219             break;
220 
221         case 8:
222             cout << "--- CARI HEWAN MILIK OWNER ---\n";
223             cout << "ID Owner: "; cin >> id1;
224             printPetsByOwner(LR, id1);
225             pauseScreen();
226             break;
227 
228         case 9:
229             cout << "--- CARI PEMILIK DARI HEWAN ---\n";
230             cout << "ID Pet: "; cin >> id1;
231             printOwnersByPet(LR, id1);
232             pauseScreen();
233             break;
234 
235         case 10:
236             cout << "--- DELETE RELATION ---\n";
237             cout << "ID Owner : "; cin >> id1;
238             cout << "ID Pet   : "; cin >> id2;
239             disconnect(LR, id1, id2);
240             pauseScreen();
241             break;
242 
```

```
        case 11:
            cout << "--- DELETE OWNER ---\n";
            cout << "ID Owner: "; cin >> id1;
            deleteOwner(LO, LR, id1);
            pauseScreen();
            break;

        case 12:
            cout << "--- DELETE PET ---\n";
            cout << "ID Pet: "; cin >> id1;
            deletePet(LP, LR, id1);
            pauseScreen();
            break;

        case 13:
            cout << "--- STATISTIK ---\n";
            cout << "Hewan tak bertuan : " << countOrphanPets(LP, LR) << endl;
            cout << "Owner tanpa hewan : " << countChildlessOwners(LO, LR) << endl;
            pauseScreen();
            break;

        case 14:
            cout << "--- EDIT RELATION ---\n";
            cout << "Old Owner ID : "; cin >> id1;
            cout << "Old Pet ID   : "; cin >> id2;
            cout << "New Owner ID : "; cin >> id3;
            cout << "New Pet ID   : "; cin >> id4;
            editRelation(LR, LO, LP, id1, id2, id3, id4);
            pauseScreen();
            break;

        case 15:
            cout << "--- EDIT DATA OWNER ---\n";
            cout << "ID Owner : "; cin >> id1;
            cout << "Nama Baru: "; cin >> nama;
            updateOwner(LO, id1, nama);
            pauseScreen();
            break;

        case 16:
            cout << "--- EDIT DATA PET ---\n";
            cout << "ID Pet    : "; cin >> id1;
            cout << "Nama Baru : "; cin >> nama;
            cout << "Jenis Baru: "; cin >> jenis;
            updatePet(LP, id1, nama, jenis);
            pauseScreen();
            break;
```

```
290
291         case 17:
292             sortOwnersByNama(LO);
293             cout << "Owner berhasil diurutkan (A-Z).\n";
294             pauseScreen();
295             break;
296
297         case 0:
298             cout << "Keluar dari program...\n";
299             break;
300
301         default:
302             cout << "Pilihan tidak valid!\n";
303             pauseScreen();
304         }
305
306     } while (choice != 0);
307
308     return 0;
309 }
310
311
```